Recursion as a pattern

Week 4, Lecture 1 · Landing the Offer: SWE Job Search for Senior CS Students

Autumn 2026 · Senior CS students

What we'll cover today

- **The recursive case:** how to spot the self-similar sub-problem
- **Base case discipline:** what makes a base case correct
- **The call stack:** how frames accumulate and unwind
- **When recursion loses to iteration:** depth limits, tail calls, the interview tradeoff

The recursive case

Recursion is a structural observation

A problem is recursive when the solution to a large instance depends on solutions to smaller instances of the same type.

- "The depth of this tree" = 1 + max(depth of left, depth of right)
- "The sum of this list" = head + sum(tail)

Three questions before writing code:

1. What is the smallest version of this problem?
2. If I had the answers for the sub-problems, how do I combine them?
3. What type does the function return? (It must be the same at every level.)

Write the recursive case in one sentence before writing any code.

Example: maximum depth of a binary tree

```
def maxDepth(root):
    if root is None:        # base case
        return 0
    left  = maxDepth(root.left)
    right = maxDepth(root.right)
    return 1 + max(left, right)
```

Recursive case: the depth of a node is 1 plus the deeper of its two subtrees.

This pattern generalizes to most tree problems where you combine results from children.

Base case discipline

The base case is the contract

A recursive function without a correct base case either:

- runs forever (infinite recursion), or
- crashes with a stack overflow once the call stack fills up

The base case must be:

1. **Reachable** on every valid input
2. **Correct** without making any further recursive calls
3. **Complete:** every possible terminal input is handled

Common base cases for trees

Situation	Base case
Node is null	return 0 (depth), null (search), true (vacuous)
Leaf node	handle without recursing
Empty list	return identity value

Missing the null check is the single most common bug in tree recursion.

Write `if root is None: return ...` first, before any other logic.
An interviewer reading your code will look there immediately.

The call stack

What happens when a function calls itself

Each call creates a new **stack frame** holding:

- The local variables for that call
- The return address (where to go when this call returns)
- The arguments passed to this call

Frames accumulate as the recursion goes deeper. They unwind as each call returns.

For a tree of height h, the stack holds at most h frames at once.

Call stack walkthrough: maxDepth on a 3-node tree


```
maxDepth(1) calls maxDepth(2)
  maxDepth(2) calls maxDepth(None) -> returns 0
  maxDepth(2) calls maxDepth(None) -> returns 0
  maxDepth(2) returns 1 + max(0, 0) = 1
maxDepth(1) calls maxDepth(3)
  maxDepth(3) calls maxDepth(None) -> returns 0
  maxDepth(3) calls maxDepth(None) -> returns 0
  maxDepth(3) returns 1 + max(0, 0) = 1
maxDepth(1) returns 1 + max(1, 1) = 2
```

Stack depth = 2 (root + one child). For a balanced tree of n nodes, max depth = O(log n). For a degenerate (linked-list) tree, max depth = O(n). **Always state call-stack space separately from output space in an interview.**

When recursion loses to iteration

Recursion has a hard ceiling

Python default recursion limit: 1,000 frames.

A degenerate binary tree with 10,000 nodes will cause a stack overflow in a naive recursive solution.

In an interview:

- State this limit if you see a potentially deep input
- Offer the iterative equivalent using an explicit stack
- Know which problems genuinely require one or the other

The iterative equivalent

Every recursive function that uses the call stack implicitly can be rewritten with an explicit stack.

```
def maxDepth_iterative(root):
    if root is None:
        return 0
    stack = [(root, 1)]
    max_depth = 0
    while stack:
        node, depth = stack.pop()
        max_depth = max(max_depth, depth)
        if node.left:
            stack.append((node.left, depth + 1))
        if node.right:
            stack.append((node.right, depth + 1))
    return max_depth
```


Same logic. No implicit call stack. Stack overflow is impossible.

Recursion vs. iteration: when to choose

Choose recursion when:

- The problem structure is naturally recursive (trees, divide-and-conquer)
- Input depth is bounded and small
- Code clarity matters more than stack safety

Choose iteration when:

- Input could be deeply nested or degenerate
- Language has a low recursion limit (Python: 1,000)
- You are implementing BFS (queue, not stack)

Take-aways

1. Identify the recursive case first: the solution to a large input in terms of smaller inputs of the same type.
2. Write the base case before any recursive calls. A missing null check is the most common tree recursion bug.
3. The call stack uses O(height) space. For a degenerate tree, height is O(n). Always state this.
4. Recursion and iteration are interchangeable. Use an explicit stack when depth could exceed the language limit.

> "These 150 problems were hand-selected to cover every major pattern you'll see in a coding interview. If you can solve all of them, you're ready."

NeetCode, NeetCode 150 Course (2022)

What's next

- **Lecture 2 (this week):** BFS and DFS in interview shape
- **Section (this week):** Pattern sprint 2, three timed mediums
- **Reading:** Week 4 reading, recursion, trees, and graph traversal
- **NeetCode roadmap:** Trees section for practice problems